Автономная некоммерческая организация высшего образования

«Московский университет «СИНЕРГИЯ»

Факультет информационных систем и технологий

ОБЗОРНАЯ РЕФЕРАТИВНАЯ РАБОТА

на тему:

«Введение в эксплуатацию программного обеспечения: deploy, развертывание и непрерывное развертывание в DevOps»

Выполнил: Лоор Идальго Жан Пьер

Группа: ДБО-301рпо

Проверил: _____

Москва, 2026

Содержание

Введение

1. Понятие введения в эксплуатацию программного обеспечения

2. Введение в эксплуатацию в жизненном цикле DevOps и CI/CD

3. Этапы deploy-процесса

4. Технологии и инструменты развертывания

5. Стратегии развертывания: rolling, blue-green, canary и feature flags

6. Требования к безопасному вводу системы в эксплуатацию

7. Риски введения в эксплуатацию и методы их снижения

8. Практический пример deploy-пайплайна веб-приложения

9. Чек-лист готовности к введению в эксплуатацию

Заключение

Список использованных источников

Введение

Современные информационные системы редко создаются как статичный продукт, который выпускается один раз и затем долго не меняется. Веб-приложения, мобильные сервисы, корпоративные платформы и облачные решения постоянно развиваются: исправляются дефекты, добавляются новые функции, обновляются зависимости, усиливаются требования безопасности и масштабируемости. Поэтому особое значение приобретает этап введения программного обеспечения в эксплуатацию.

В рамках данной работы термин «введение в эксплуатацию» рассматривается как процесс deploy — развертывание программного продукта в целевой среде с последующим контролем его работоспособности. Это не только копирование файлов на сервер, а управляемая инженерная процедура, включающая подготовку артефактов, настройку окружения, проверку качества, миграции данных, мониторинг, документирование, обратную связь и возможность безопасного отката.

Актуальность темы связана с тем, что именно этап развертывания часто становится границей между разработкой и реальной ценностью для пользователя. Даже качественно написанный код не приносит пользы, если команда не может стабильно, воспроизводимо и безопасно ввести его в рабочую среду. В литературе по непрерывной интеграции подчеркивается, что процессы компиляции, проверки, инспекции кода и развертывания могут быть автоматизированы и запускаться после каждого изменения программного обеспечения [1].

Цель работы — рассмотреть основы введения программного обеспечения в эксплуатацию как части методологии DevOps и CI/CD, определить ключевые этапы deploy-процесса, инструменты, стратегии, риски и критерии готовности системы к продуктивной эксплуатации.

Для достижения цели поставлены следующие задачи: раскрыть понятие введения в эксплуатацию; показать его связь с DevOps, CI/CD и непрерывным развертыванием; описать типовой deploy-пайплайн; проанализировать инструменты и стратегии развертывания; сформировать чек-лист готовности к выпуску; привести практический пример процесса ввода веб-приложения в эксплуатацию.

1. Понятие введения в эксплуатацию программного обеспечения

Введение в эксплуатацию программного обеспечения — это совокупность организационных и технических действий, в результате которых разработанная система начинает использоваться в целевой среде. В узком смысле данный этап можно назвать развертыванием, или deploy. В широком смысле он включает подготовку инфраструктуры, настройку конфигурации, управление версиями, проверку работоспособности, информирование участников проекта и дальнейшее сопровождение.

Развертывание отличается от простой установки тем, что оно должно быть воспроизводимым. Если команда может выполнить deploy только вручную, по памяти или через набор неформальных действий, то процесс становится зависимым от конкретного специалиста и плохо контролируется. В профессиональной разработке предпочтение отдается автоматизированным сценариям, декларативным конфигурациям и заранее описанным процедурам.

Введение в эксплуатацию можно рассматривать как переход программного продукта из состояния «разработано» в состояние «доступно пользователям». Этот переход должен отвечать нескольким требованиям: предсказуемость, повторяемость, контролируемость, безопасность, наблюдаемость и возможность отката. Если хотя бы одно из этих требований не выполняется, релиз становится источником повышенного риска.

Ключевая особенность современного подхода состоит в том, что deploy перестает быть редким и опасным событием. В DevOps-культуре развертывание стремятся сделать регулярной инженерной операцией. Чем чаще и меньше по объему изменения, тем проще их проверять, выпускать и при необходимости откатывать.

Таблица 1 — Различие между установкой, развертыванием и введением в эксплуатацию

Понятие	Содержание	Типичные действия	Результат
Установка	Размещение программы или компонента на устройстве	Копирование файлов, запуск инсталлятора	Приложение физически установлено
Развертывание (deploy)	Доставка версии в целевую среду	Сборка, публикация артефакта, настройка окружения, запуск	Версия работает в нужной среде
Введение в эксплуатацию	Организованный переход к промышленному использованию	Deploy, проверка, мониторинг, документация, поддержка, откат	Система доступна пользователям и контролируется командой

2. Введение в эксплуатацию в жизненном цикле DevOps и CI/CD

DevOps объединяет разработку, тестирование, эксплуатацию и обратную связь в единый непрерывный процесс. В таком подходе ответственность команды не заканчивается написанием кода. Команда отвечает за то, чтобы код был собран, проверен, безопасно доставлен в целевую среду, наблюдаем в эксплуатации и быстро исправляем при возникновении проблем.

CI/CD является практической основой DevOps. CI (Continuous Integration) обеспечивает частую интеграцию изменений и автоматическую проверку качества. CD может пониматься как Continuous Delivery — непрерывная

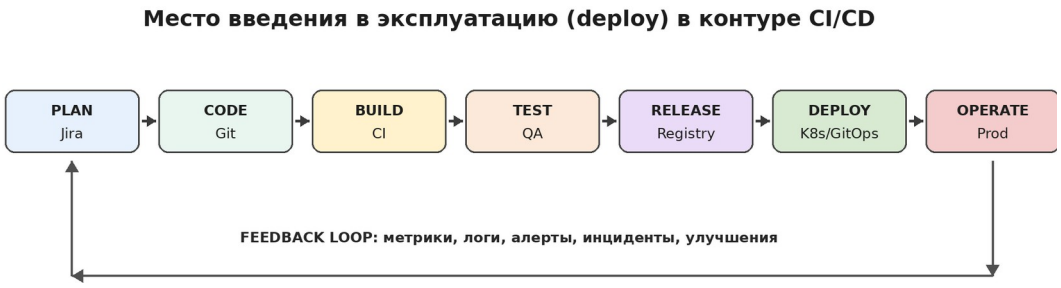
доставка, при которой система всегда готова к релизу, либо как Continuous Deployment — непрерывное развертывание, когда успешные изменения автоматически проходят в целевую среду.

В рекомендованном преподавателем материале CICD представлена схема цикла, где этап deploy расположен после release и связан с дальнейшей эксплуатацией и мониторингом. Вокруг цикла указаны типичные инструменты: GitHub и GitLab для кода, Gradle, Bazel и Webpack для сборки, Jest, JUnit и Playwright для тестирования, Jenkins и Buildkite для автоматизации, Docker, Kubernetes, Terraform, AWS Lambda, Prometheus, Datadog и Argo CD для инфраструктуры, эксплуатации и мониторинга [2].

Книга П. Дюваля, С. Матиаса и Э. Гловера выделяет «непрерывное развертывание» как часть полнофункциональной системы непрерывной интеграции. В оглавлении отдельная глава посвящена непрерывному развертыванию, включая маркировку элементов в хранилище, чистоту среды, маркировку каждого построения, запуск проверок, отчеты обратной связи и возможность отката выпуска [1].

Таким образом, введение в эксплуатацию следует рассматривать не как финальный ручной шаг, а как контролируемый участок CI/CD-пайплайна. Оно соединяет инженерные практики сборки и тестирования с реальной эксплуатацией системы.

Рисунок 1 — Место введения в эксплуатацию в контуре CI/CD



Источник: составлено автором на основе материалов CI/CD и DevOps.

3. Этапы deploy-процесса

Типовой deploy-процесс начинается не в момент нажатия кнопки «развернуть». Он формируется заранее: на этапе проектирования архитектуры, выбора инфраструктуры, настройки репозитория, написания тестов и определения правил выпуска. Чем раньше команда описывает правила эксплуатации, тем меньше неожиданностей возникает при релизе.

В общем виде процесс введения программного обеспечения в эксплуатацию можно представить как последовательность этапов: подготовка релиза, сборка артефакта, проверка качества, подготовка среды, развертывание, post-deploy проверка, мониторинг и откат при необходимости. Такой подход позволяет контролировать как технические, так и организационные аспекты выпуска.

Дюваль и соавторы описывают типичное развертывание работоспособного программного обеспечения через шесть действий: маркировка элементов в хранилище, поддержание чистоты среды, построение непосредственно из хранилища, проверка на уровнях копии рабочей среды, создание отчетов обратной связи и откат выпуска при необходимости [1]. Эти действия остаются актуальными и в современных инструментах, хотя вместо

Ant и Tomcat сегодня часто применяются контейнеры, Kubernetes, GitHub Actions, GitLab CI, Argo CD и облачные сервисы.

Важнейшим элементом является создание артефакта. Артефактом может быть исполняемый файл, пакет, Docker-образ, Helm chart, архив статического сайта или набор конфигураций инфраструктуры. Артефакт должен быть неизменяемым: однажды собранная версия не должна пересобираться под тем же номером, иначе невозможно гарантировать воспроизводимость релиза.

После публикации артефакта выполняется развертывание в целевую среду. Обычно выделяют несколько сред: development, test, staging и production. Среда staging должна максимально соответствовать production, потому что именно в ней удобно проводить финальные проверки до передачи пользователям.

Таблица 2 — Основные этапы введения программного обеспечения в эксплуатацию

Этап	**Цель**	**Действия**	**Инструменты**	**Контрольный результат**
Подготовка релиза	Определить версию и состав изменений	Release notes, tag, changelog	Git, Jira, Confluence	Понятен состав выпуска
Сборка артефакта	Получить воспроизводимый результат	Build, packaging, image build	Gradle, Maven, Webpack, Docker	Создан артефакт версии
Проверка качества	Не допустить дефектный выпуск	Unit, integration, e2e, security checks	JUnit, Jest, Playwright, SAST	Проверки прошли успешно
Подготовка среды	Обеспечить совместимое окружение	IaC, secrets, variables, DB migrations	Terraform, Kubernetes, Vault	Среда готова
Развертывание	Доставить версию пользователям	Rolling/ canary/blue-green deploy	Kubernetes, Argo CD, Jenkins	Версия запущена
Post-deploy проверка	Подтвердить работоспособность	Smoke tests, health checks	CI/CD runner, monitoring	Система отвечает корректно

Мониторинг и откат	Контролировать последствия релиза	Alerts, logs, rollback	Prometheus, Grafana, Datadog	Риски управляемы

4. Технологии и инструменты развертывания

Инструменты введения в эксплуатацию можно разделить на несколько групп. Каждая группа отвечает за отдельный аспект процесса: хранение исходного кода, сборку, тестирование, публикацию артефактов, контейнеризацию, управление инфраструктурой, оркестрацию, GitOps и мониторинг.

Системы контроля версий, такие как GitHub и GitLab, используются как источник правды для исходного кода и конфигурации. На их основе запускаются pipeline-процессы: сборка, тестирование, публикация и развертывание. Документация GitHub Actions описывает платформу как средство автоматизации workflows, включая CI/CD, сборку, тестирование и deployment pipeline [4].

Контейнеризация является одной из ключевых технологий современного deploy. Docker позволяет упаковать приложение и его зависимости в образ, который затем можно запустить в одинаковом виде в разных средах. Официальная документация Docker описывает процессы сборки, тегирования и публикации образов в registry [5].

Kubernetes используется для управления контейнеризованными приложениями. Deployment в Kubernetes описывает желаемое состояние приложения и позволяет выполнять декларативные обновления Pod и ReplicaSet [6]. При rolling update новая версия вводится постепенно, что снижает риск простоя, а команда kubectl rollout undo позволяет откатиться к предыдущему rollout при необходимости [7].

Argo CD реализует GitOps-подход: конфигурации приложений и окружений хранятся декларативно в Git, а система автоматически

синхронизирует фактическое состояние кластера с желаемым состоянием. Официальная документация определяет Argo CD как декларативный инструмент continuous delivery для Kubernetes [8].

Таблица 3 — Инструменты, применяемые при введении в эксплуатацию

Группа	Инструменты	Назначение	Роль в deploy
Контроль версий	GitHub, GitLab	Хранение кода и конфигураций	Источник версии и триггер pipeline
CI/CD серверы	Jenkins, GitHub Actions, GitLab CI	Автоматизация workflow	Сборка, тесты, доставка
Сборка	Gradle, Maven, Webpack, Bazel	Создание артефакта	Подготовка пакета релиза
Тестирование	JUnit, Jest, Playwright	Проверка качества	Блокировка дефектного релиза
Контейнеризация	Docker, BuildKit	Создание образов	Единая среда запуска
Registry	Docker Hub, GitHub Packages, Harbor	Хранение образов	Доставка immutable artifact
Оркестрация	Kubernetes	Управление контейнерами	Rolling update, scaling, health checks
GitOps	Argo CD	Синхронизация Git и кластера	Автоматизированный и проверяемый deploy
IaC	Terraform	Описание инфраструктуры как кода	Воспроизводимая среда
Мониторинг	Prometheus, Grafana, Datadog	Метрики, логи, алерты	Контроль после релиза

5. Стратегии развертывания: rolling, blue-green, canary и feature flags

Стратегия развертывания определяет, каким образом новая версия приложения будет введена в эксплуатацию. Выбор стратегии зависит от критичности системы, нагрузки, требований к доступности, архитектуры приложения, зрелости тестов и возможностей инфраструктуры.

Rolling update — постепенное замещение старых экземпляров приложения новыми. Этот подход широко применяется в Kubernetes. Его преимущество заключается в том, что новая версия вводится без полной остановки сервиса.

Недостаток состоит в том, что некоторое время в системе могут одновременно работать старая и новая версии.

Blue-green deployment предполагает наличие двух идентичных окружений. Одно окружение обслуживает пользователей, второе содержит новую версию. После проверки трафик переключается на новое окружение. Если возникают проблемы, можно быстро вернуть трафик назад. Такая стратегия удобна для критичных сервисов, но требует больше ресурсов.

Canary deployment означает постепенную передачу небольшой части трафика на новую версию. Например, сначала 5 % пользователей получают обновление, затем 25 %, затем 50 % и только после успешного мониторинга — 100 %. Такой подход позволяет обнаружить проблемы на ограниченной аудитории.

Feature flags позволяют отделить развертывание кода от включения функциональности. Код может быть доставлен в production, но новая возможность будет выключена до момента, когда команда решит включить ее для определенной группы пользователей. Это снижает риск крупных релизов и упрощает A/B-тестирование.

Таблица 4 — Сравнение стратегий deploy

Стратегия	**Суть**	**Преимущества**	**Недостатки**	**Когда применять**
Rolling update	Постепенная замена экземпляров	Нет полной остановки	Временная работа двух версий	Большинство веб-сервисов
Blue-green	Два окружения и переключение трафика	Быстрый откат	Высокая стоимость ресурсов	Критичные релизы
Canary	Новая версия для части пользователей	Раннее выявление дефектов	Нужны метрики и маршрутизация	Высоконагруженные сервисы
Feature flags	Функция включается независимо от deploy	Гибкий контроль релиза	Нужна дисциплина удаления flags	Пошаговый выпуск функций

6. Требования к безопасному вводу системы в эксплуатацию

Безопасный ввод в эксплуатацию требует не только технической готовности приложения, но и управляемости всего процесса. Команда должна понимать, какая версия выпускается, какие изменения она содержит, какие проверки уже пройдены, какие зависимости обновлены и каким образом будет выполнен откат.

Первое требование — наличие контроля версий. Исходный код, конфигурации, инфраструктурные манифесты, миграции базы данных и pipeline-скрипты должны храниться в репозитории. Это обеспечивает трассируемость изменений и возможность восстановления состояния.

Второе требование — автоматизированные проверки. Перед вводом в эксплуатацию необходимо выполнить unit-тесты, интеграционные тесты, e2e-тесты, статический анализ, проверку зависимостей и, при необходимости, нагрузочные тесты. В книге по CI подчеркивается необходимость запускать все проверки перед переходом программного обеспечения на следующий этап рабочего процесса [1].

Третье требование — управляемая конфигурация. Конфигурации не должны быть жестко встроены в код. Для разных сред используются переменные окружения, secret management, конфигурационные карты, хранилища секретов и инфраструктура как код.

Четвертое требование — наблюдаемость. После deploy команда должна видеть состояние системы через метрики, журналы, трассировки и алерты. Без мониторинга невозможно быстро понять, успешен ли релиз и не ухудшил ли он пользовательский опыт.

Пятое требование — заранее подготовленный rollback-план. Откат не должен проектироваться уже после инцидента. Он должен быть частью

релизного процесса: сохранение предыдущей версии, метки в репозитории, резервные копии, совместимость миграций и команда для возврата.

7. Риски введения в эксплуатацию и методы их снижения

Введение системы в эксплуатацию связано с рядом рисков. Наиболее распространенные проблемы: несовместимость версии с окружением, дефекты, не выявленные тестами, ошибка конфигурации, неудачная миграция базы данных, отсутствие мониторинга, ручная ошибка при выполнении инструкций, невозможность быстрого отката.

В традиционных проектах релиз часто был редким и крупным событием, из-за чего риск возрастал. Современный DevOps-подход стремится уменьшить размер изменений и автоматизировать повторяемые действия. Чем меньше ручных операций, тем ниже вероятность случайной ошибки.

В книге Дюваля и соавторов среди рисков CI отдельно выделяется отсутствие развертываемого программного обеспечения, позднее выявление дефектов, плохой контроль проекта и низкое качество программного обеспечения [1]. Все эти риски напрямую связаны с вводом в эксплуатацию: если код нельзя надежно собрать, проверить и доставить, то система не готова к промышленному использованию.

Ключевые методы снижения рисков: автоматизация pipeline, immutable artifacts, staging-среда, обязательные тесты, deployment gates, мониторинг, canary-релизы, документированные rollback-процедуры и регулярное проведение post-release анализа.

Таблица 5 — Риски deploy и меры контроля

Риск	Проявление	Последствие	Метод снижения
Ошибка конфигурации	Неверные переменные, секреты, адреса сервисов	Сервис не запускается	IaC, config review, secrets management

Дефект в новой версии	Ошибки после релиза	Падение функций	Тесты, canary, feature flags
Проблема базы данных	Неудачная миграция	Потеря данных или простой	Backups, reversible migrations, dry run
Нет мониторинга	Команда поздно узнает о сбое	Рост времени простоя	Метрики, логи, alerts, SLO
Ручная ошибка	Неверная команда или файл	Нестабильный релиз	Автоматизация, checklist, approval
Нет отката	Нельзя вернуть прошлую версию	Долгий инцидент	Rollback plan, tags, immutable artifacts

8. Практический пример deploy-пайплайна веб-приложения

Рассмотрим пример введения в эксплуатацию веб-приложения, которое хранится в GitHub, собирается в Docker-образ, публикуется в registry и разворачивается в Kubernetes. Для управления состоянием кластера применяется Argo CD, а мониторинг выполняется через Prometheus и Grafana.

Такой пример отражает современную практику: исходный код и конфигурации хранятся в Git, pipeline автоматически запускается при изменении, создается неизменяемый артефакт, а фактическое состояние production-среды синхронизируется с декларативным описанием.

1. Разработчик создает ветку feature и вносит изменения в код.
2. После pull request автоматически запускаются сборка и тесты.
3. При успешной проверке изменения сливаются в main.
4. CI-система собирает Docker-образ и присваивает ему уникальный tag.
5. Образ публикуется в container registry.
6. В Git-репозитории инфраструктуры обновляется версия образа.
7. Argo CD обнаруживает изменение и синхронизирует Kubernetes-кластер.
8. Kubernetes выполняет rolling update и проводит health checks.
9. После deploy выполняются smoke tests.
10. Prometheus и Grafana отслеживают метрики; при критических отклонениях выполняется rollback.

Пример упрощенного GitHub Actions workflow для сборки и публикации Docker-образа:

```
name: build-and-publish
on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t registry.example.com/app:${{ github.sha }} .
      - name: Run tests
        run: npm test
      - name: Push Docker image
        run: docker push registry.example.com/app:${{ github.sha }}
```

Данный пример является учебным и показывает общую логику процесса. В реальном проекте необходимо добавить авторизацию в registry, управление секретами, сканирование образов, разделение сред, ручные approvals для production и проверки безопасности.

9. Чек-лист готовности к введению в эксплуатацию

Для практического применения темы целесообразно использовать чек-лист готовности. Он помогает команде убедиться, что система готова к выпуску не только с точки зрения кода, но и с точки зрения эксплуатации.

- Версия приложения имеет уникальный номер, tag или commit hash.
- Исходный код, конфигурации и инфраструктурные файлы находятся в системе контроля версий.
- Сборка выполняется автоматически и воспроизводимо.
- Все обязательные тесты прошли успешно.
- Создан и опубликован неизменяемый артефакт релиза.
- Секреты и переменные окружения настроены безопасно.
- Миграции базы данных проверены на тестовой или staging-среде.
- Настроены health checks, readiness и liveness checks.

- Настроены метрики, логи и алерты.
- Подготовлен rollback-план и проверена возможность отката.
- Ответственные лица знают порядок действий при инциденте.
- Документация релиза и краткое описание изменений подготовлены.

Заключение

Введение программного обеспечения в эксплуатацию является одним из ключевых этапов жизненного цикла программного продукта. В современной разработке этот этап нельзя сводить к ручному копированию файлов на сервер. Он представляет собой комплексный управляемый процесс, включающий сборку, тестирование, подготовку окружения, развертывание, мониторинг, обратную связь и возможность отката.

Методология DevOps и технологии CI/CD позволяют сделать deploy более надежным и предсказуемым. Благодаря автоматизации команда получает возможность выпускать изменения чаще, уменьшать размер релизов, быстрее обнаруживать ошибки и снижать влияние человеческого фактора. Непрерывное развертывание становится кульминацией практик CI/CD, так как связывает разработку с реальной эксплуатацией системы.

В работе были рассмотрены основные этапы deploy-процесса, инструменты, стратегии развертывания и риски. Особое внимание уделено таким технологиям, как Docker, Kubernetes, GitHub Actions, Jenkins, Terraform, Argo CD, Prometheus и Datadog. Были описаны rolling update, blue-green deployment, canary deployment и feature flags как способы снижения риска релиза.

Главный вывод заключается в том, что успешное введение в эксплуатацию требует не только инструментов, но и дисциплины процесса: контроля версий, автоматизированных проверок, прозрачной конфигурации, мониторинга, документации и готовности к откату. Только при соблюдении

этих условий программное обеспечение можно выпускать быстро, безопасно и с высоким уровнем доверия к результату.

Список использованных источников

1. Дюваль П. М., Матиас С., Гловер Э. Непрерывная интеграция: улучшение качества программного обеспечения и снижение риска. — М.: Вильямс, 2008. — 240 с.
2. CICD: подборка материалов по CI/CD, инструментам сборки, тестирования, развертывания, мониторинга и GitOps. — Материалы, предоставленные преподавателем.
3. Fowler M. Continuous Integration. — URL: https://martinfowler.com/articles/continuousIntegration.html
4. GitHub Docs. Understanding GitHub Actions. — URL: https://docs.github.com/articles/getting-started-with-github-actions
5. GitHub Docs. Continuous deployment. — URL: https://docs.github.com/en/actions/get-started/continuous-deployment
6. Docker Docs. Build, tag, and publish an image. — URL: https://docs.docker.com/get-started/docker-concepts/building-images/build-tag-and-publish-an-image/
7. Kubernetes Documentation. Deployments. — URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
8. Kubernetes Documentation. Performing a Rolling Update. — URL: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/
9. Kubernetes Documentation. kubectl rollout undo. — URL: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/kubectl_rollout_undo/
10. Argo CD Documentation. Declarative GitOps CD for Kubernetes. — URL: https://argo-cd.readthedocs.io/en/stable/
11. Jenkins Documentation. — URL: https://www.jenkins.io/doc/
12. Terraform Documentation. — URL: https://developer.hashicorp.com/terraform/docs

13. Prometheus Documentation. — URL: https://prometheus.io/docs/introduction/overview/
14. Gradle User Manual. — URL: https://docs.gradle.org/current/userguide/userguide.html
15. Jest Documentation. — URL: https://jestjs.io/docs/getting-started
16. Playwright Documentation. — URL: https://playwright.dev/docs/intro
17. JUnit 5 User Guide. — URL: https://junit.org/junit5/docs/current/user-guide/
18. Habr. Основы CI/CD. Знакомство с Jenkins. — URL: https://habr.com/ru/companies/slurm/articles/691876/
19. Habr. GitOps-платформа на базе Argo CD. — URL: https://habr.com/ru/companies/kts/articles/723760/